Aula 06 — Estruturas de repetição com while

Repetindo instruções, controlando laços e construindo menus simples

Dieimes Nunes

Retomando a Aula 05

Na aula anterior, trabalhamos com decisões mais completas.

Vimos situações com:

- `if`;
- `else`;
- `elif`;
- condições compostas com `and` e `or`;
- condições aninhadas;
- menu simples;
- teste de mesa com tomada de decisão.

Exemplo:

```
opcao = int(input("Digite uma opção: "))

if opcao == 1:
    print("Cadastrar")
elif opcao == 2:
    print("Consultar")
elif opcao == 3:
    print("Sair")
else:
    print("Opção inválida")
```

Hoje vamos dar um passo importante: fazer o programa **repetir ações**.

O problema da aula

Imagine que precisamos mostrar uma mensagem cinco vezes.

Poderíamos escrever assim:

```
print("Bem-vindo ao curso")
print("Bem-vindo ao curso")
print("Bem-vindo ao curso")
print("Bem-vindo ao curso")
print("Bem-vindo ao curso")
```

Funciona, mas não é uma boa solução.

E se fossem 50 vezes?

E se fossem 500 vezes?

E se a quantidade dependesse do usuário?

Para isso, usamos **estruturas de repetição**.

O que é uma estrutura de repetição?

Uma estrutura de repetição permite executar um mesmo bloco de comandos várias vezes.

Ela também é chamada de:

- laço;
- loop;
- repetição;
- estrutura iterativa.

A ideia é simples:

```
Enquanto uma condição for verdadeira:
    execute um conjunto de comandos
```

Quando a condição deixar de ser verdadeira, a repetição termina.

Repetição no cotidiano

Usamos repetição o tempo todo.

Exemplos:

- enquanto o arroz não estiver pronto, continuar cozinhando;
- enquanto houver louça suja, lavar a próxima peça;
- enquanto a senha estiver errada, pedir novamente;
- enquanto houver alunos para chamar, registrar presença;
- enquanto o estoque tiver produtos, continuar vendendo;
- enquanto o usuário não escolher sair, manter o sistema aberto.

Atividade 1 — Repetições do cotidiano

No caderno, escolha uma situação do cotidiano que envolve repetição.

Exemplos:

- lavar roupas;
- contar alunos presentes;
- verificar produtos no estoque;
- tentar senha do celular;
- escovar os dentes;
- abastecer um veículo.

Escreva no formato:

```
Enquanto _____________________________:
    faça _____________________________
```

Depois, responda:

1. Qual é a condição para continuar repetindo?
2. Qual é a condição para parar?
3. O que acontece em cada repetição?

A estrutura while

Em Python, usamos `while` quando queremos repetir comandos **enquanto uma condição for verdadeira**.

Estrutura básica:

```
while condição:
    comando_1
    comando_2
    comando_3
```

O bloco repetido fica identificado pela **indentação**.

A condição deve gerar um resultado lógico:

- `True`, quando a repetição continua;
- `False`, quando a repetição termina.

Primeiro exemplo com while

Vamos contar de 1 até 5.

```
contador = 1

while contador <= 5:
    print(contador)
    contador = contador + 1
```

Resultado esperado:

```
1
2
3
4
5
```

Esse programa possui três partes importantes:

1. início do contador;
2. condição de repetição;
3. atualização do contador.

Entendendo o contador

Neste código:

```
contador = 1

while contador <= 5:
    print(contador)
    contador = contador + 1
```

A variável `contador` controla a repetição.

Ela começa com o valor `1`.

A cada volta do laço, ela aumenta uma unidade.

Quando o contador chega a `6`, a condição `contador <= 5` se torna falsa.

Então o `while` termina.

Teste de mesa do while

Volta	contador antes	condição `contador <= 5`	saída	contador depois
1	1	Verdadeira	1	2
2	2	Verdadeira	2	3
3	3	Verdadeira	3	4
4	4	Verdadeira	4	5
5	5	Verdadeira	5	6
6	6	Falsa	não executa	6

O teste de mesa ajuda a entender o comportamento do código antes mesmo de executar.

Atividade 2 — Teste de mesa

Analise o código abaixo sem executar no computador.

```
numero = 2

while numero <= 10:
    print(numero)
    numero = numero + 2
```

No caderno, responda:

1. Qual será a primeira saída?
2. Qual será a última saída?
3. Quantas vezes o laço será executado?
4. Qual será o valor final da variável `numero`?

Laço infinito

Um laço infinito acontece quando a condição do `while` nunca se torna falsa.

Exemplo:

```
contador = 1

while contador <= 5:
    print(contador)
```

O problema é que o contador nunca muda.

Ele sempre continua valendo `1`.

Portanto, a condição `contador <= 5` será sempre verdadeira.

Esse tipo de erro é comum no início.

Como evitar laços infinitos

Antes de criar um `while`, pense em três perguntas:

1. Qual variável controla a repetição?
2. Qual é a condição para continuar?
3. Em que momento essa variável será alterada?

Modelo seguro:

```
contador = 1

while contador <= 5:
    print(contador)
    contador = contador + 1
```

Se a variável de controle não for alterada, o programa pode nunca parar.

Atividade 3 — Corrigindo laço infinito

O código abaixo possui um problema.

```
x = 1

while x <= 3:
    print("Executando...")
```

No caderno:

1. Explique qual é o erro.
2. Reescreva o código corrigido.
3. Diga qual será a saída esperada.

Depois, teste a versão corrigida no Python.

Contagem crescente

Para contar em ordem crescente, aumentamos a variável de controle.

Exemplo:

```
contador = 1

while contador <= 10:
    print(contador)
    contador = contador + 1
```

Também poderíamos usar:

```
contador += 1
```

Esse comando é uma forma abreviada de escrever:

```
contador = contador + 1
```

Contagem decrescente

Para contar em ordem decrescente, diminuímos a variável de controle.

Exemplo:

```
contador = 10

while contador >= 1:
    print(contador)
    contador = contador - 1
```

Forma abreviada:

```
contador -= 1
```

Esse tipo de repetição aparece em:

- contagem regressiva;
- cronômetros;
- tentativas restantes;
- controle de limite.

Atividade 4 — Contagens

Crie três programas diferentes.

Programa 1

Mostrar os números de 1 até 20.

Programa 2

Mostrar os números de 20 até 1.

Programa 3

Mostrar apenas os números pares de 2 até 30.

Antes de programar, escreva no caderno:

- valor inicial;
- condição de parada;
- forma de atualização.

Contador e acumulador

Em estruturas de repetição, usamos muito duas ideias:

Contador

Conta quantas vezes algo aconteceu.

```
quantidade = quantidade + 1
```

Acumulador

Soma valores ao longo da repetição.

```
total = total + valor
```

O contador normalmente aumenta de 1 em 1.

O acumulador recebe valores variados.

Exemplo com acumulador

Vamos somar os números de 1 até 5.

```
contador = 1
total = 0

while contador <= 5:
    total = total + contador
    contador = contador + 1

print(f"Total: {total}")
```

Resultado esperado:

```
Total: 15
```

O programa soma:

```
1 + 2 + 3 + 4 + 5 = 15
```

Entendendo o acumulador

No início:

```
total = 0
```

A variável `total` começa zerada porque ainda não somamos nada.

A cada repetição:

```
total = total + contador
```

O valor atual do contador é somado ao total.

Ao final, o total contém a soma acumulada.

Atividade 5 — Soma com while

Crie um programa que solicite cinco números ao usuário e mostre a soma final.

Modelo de funcionamento:

```
Digite um número: 10
Digite um número: 5
Digite um número: 8
Digite um número: 2
Digite um número: 1
Soma final: 26
```

Dicas:

Resolução comentada

```python
contador = 1
soma = 0

while contador <= 5:
    numero = float(input("Digite um número: "))
    soma = soma + numero
    contador = contador + 1

print(f"Soma final: {soma}")
```

Observe:

- `contador` controla quantas vezes o usuário digita;
- `soma` guarda o total acumulado;
- `numero` recebe o valor digitado em cada repetição.

Calculando média com repetição

Podemos adaptar o programa anterior para calcular média.

```
contador = 1
soma = 0

while contador <= 4:
    nota = float(input(f"Digite a {contador}ª nota: "))
    soma = soma + nota
    contador = contador + 1

media = soma / 4

print(f"Média: {media:.2f}")
```

Agora o programa repete a leitura das notas e calcula o resultado final.

Atividade 6 — Média de notas

Crie um programa que leia quatro notas de um aluno e mostre:

- a soma das notas;
- a média final;
- a situação do aluno.

Regras:

```
Média maior ou igual a 7: aprovado
Média menor que 7: reprovado
```

Esse exercício une:

- entrada de dados;
- conversão com `float()`;
- repetição com `while`;
- acumulador;
- estrutura condicional.

Repetindo até uma condição mudar

Nem sempre sabemos quantas vezes a repetição deve acontecer.

Exemplo: pedir uma senha até o usuário acertar.

```
senha = input("Digite a senha: ")

while senha != "1234":
    print("Senha incorreta")
    senha = input("Digite a senha novamente: ")

print("Acesso liberado")
```

Nesse caso, a repetição depende da resposta do usuário.

Por que a entrada aparece antes e dentro do while?

No exemplo da senha:

```
senha = input("Digite a senha: ")

while senha != "1234":
    print("Senha incorreta")
    senha = input("Digite a senha novamente: ")
```

A primeira entrada é necessária para testar a condição pela primeira vez.

A segunda entrada, dentro do laço, permite que o usuário tente novamente.

Sem essa segunda entrada, o programa ficaria preso em laço infinito.

Atividade 7 — Senha com repetição

Crie um programa que peça uma senha até o usuário acertar.

Regras:

- senha correta: `senac123`;
- enquanto a senha estiver incorreta, mostrar `Senha incorreta`;
- quando acertar, mostrar `Acesso permitido`.

Desafio adicional:

Menu com while

O `while` também pode manter um menu funcionando até o usuário escolher sair.

Exemplo:

```
opcao = 0

while opcao != 3:
    print("1 - Cadastrar")
    print("2 - Consultar")
    print("3 - Sair")

    opcao = int(input("Escolha uma opção: "))

    if opcao == 1:
        print("Cadastrar selecionado")
    elif opcao == 2:
        print("Consultar selecionado")
    elif opcao == 3:
        print("Encerrando...")
```

Entendendo o menu

Neste menu, a variável `opcao` controla a repetição.

```
while opcao != 3:
```

Enquanto a opção for diferente de 3, o menu continua aparecendo.

Quando o usuário digita 3, a condição se torna falsa.

Então o programa encerra a repetição.

Esse modelo será muito importante para projetos maiores.

Atividade 8 — Menu de sistema

Crie um menu com as opções:

```
1 - Somar dois números
2 - Verificar se um número é par ou ímpar
3 - Mostrar uma mensagem de boas-vindas
4 - Sair
```

Regras:

- o menu deve repetir enquanto o usuário não escolher sair;
- cada opção deve executar uma ação diferente;
- opção inválida deve mostrar uma mensagem de erro.

Cuidados com while

Erros comuns:

- esquecer de atualizar a variável de controle;
- criar uma condição que nunca se torna falsa;
- atualizar a variável no lugar errado;
- confundir `=` com `==`;
- esquecer a indentação;
- usar conversão incorreta com `int()` ou `float()`;
- não testar diferentes entradas.

Onde estamos no curso

Até agora, já trabalhamos com:

- lógica e algoritmos;
- entrada, processamento e saída;
- variáveis e tipos de dados;
- operadores e expressões;
- estruturas condicionais;
- condições encadeadas;
- introdução às estruturas de repetição.

A partir de agora, os programas começam a ficar mais próximos de sistemas reais, porque conseguem decidir e repetir ações.

Desafio final da aula

Crie um programa de controle de caixa simples.

O sistema deve apresentar o menu:

```
1 - Registrar venda
2 - Mostrar total vendido
3 - Sair
```

Regras:

- ao registrar venda, pedir o valor da venda;
- acumular o total vendido;
- mostrar o total quando o usuário escolher a opção 2;
- encerrar somente na opção 3.

Fechamento

Hoje aprendemos que o `while` permite repetir comandos enquanto uma condição for verdadeira.

Também vimos:

- contador;
- acumulador;
- laço infinito;
- repetição com quantidade definida;
- repetição até o usuário acertar;
- menu com repetição.

Na próxima aula, vamos conhecer outra forma de repetição: o `for`, muito usado quando sabemos a quantidade de repetições.